

Real-world (a cold design-review interview on your finished capstone) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried this against your own finished capstone. Each hint has 2 depth levels: **Basic** (the plain framing — how this differs from a walkthrough) and **Intermediate** (the real technique — full-system coverage, not one prepared story, then the senior-level judgment call — knowing which choices are truly defensible versus which were just what you happened to build, and admitting the difference). Read Basic first even if you're confident — it's the fastest way to see exactly what each deeper level adds.

- [Hint 1 — The idea, and how this differs from Doc17](#)
- [Hint 2 — The plan, and a worked example](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and how this differs from Doc17

Basic Version

Doc17's exercise let you pick the order — problem, then design, then one decision, then one limitation. This exercise doesn't. The interviewer picks what to ask about, can jump to any piece of your system at any time, and can push back on a choice you never flagged as debatable in the first place.

Before you start, don't prepare a script. Instead, make sure you can talk about *every* major piece of your capstone, not just the one decision you're proudest of — because you don't get to choose which one comes up first.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

This is Doc16's design defense and Doc17's project walkthrough, combined and then made harder in one specific way: **coverage**, not depth on one topic, is what's actually being tested. A cold reviewer with "full authority to challenge any decision" ranges freely across everything in this document's "What You'll Design and Build" list — agents, tools, workflow, state, RAG, database, APIs, login/permissions, testing, monitoring, deployment, cost — and can move between them in any order, including circling back to something you thought you'd already closed out.

The practical skill: for each subsystem, be ready with the same shape Doc16 and Doc17 already trained — what it is, the specific requirement it satisfies, the strongest real alternative, and what that alternative would have cost you. The difference here is you need this ready for *all* of them at once, not one rehearsed answer for your favorite piece.

Go through your own capstone's build list now and rate your own readiness on each piece — honestly — before Hint 2.

When the reviewer finds a seam:

The senior-level trap in this exercise isn't forgetting a piece of your system — it's treating every choice as if it were a deeply reasoned decision, when some of them genuinely weren't. Some pieces of your capstone were built because the requirements demanded them specifically. Others were built because that was simply the first reasonable option you reached for, and you never seriously considered an alternative. A design-review interviewer at real senior-architect depth can usually tell the difference between the two from how you answer — a genuine decision has a specific cost you can name; a default has a vague one, or none.

The senior move is being willing to say, out loud, "that one was arbitrary — I'd pick differently with more time, and here's what I'd pick" for the pieces that actually were arbitrary, instead of reflexively defending everything as if it were principled. Over-defending an arbitrary choice is a worse signal than admitting it plainly, because it suggests you can't tell the difference between a decision and a default — which is exactly the judgment a senior architect is being hired to have.

There's a second, harder version of this same trap: two of your decisions can be in tension with each other — a principle you invoked to defend one choice quietly contradicts a different choice elsewhere in the same system. A cold interviewer who has looked at your whole design, not just the piece you're currently discussing, may find that seam and ask you to reconcile it live. The honest move is the same one Doc16 and Doc17's harder pushback scenarios already train: name the inconsistency plainly, say which side you'd actually fix, and don't invent a reason on the spot that makes both halves sound intentional when they weren't.

Before Hint 2, go back through your own build list and mark each piece as either "genuine decision — I can name the specific requirement and the alternative's real cost" or "default — I'd reconsider this with more time." Being honest about which pile each piece belongs in is the actual preparation for this exercise.

Difference between Basic and Intermediate: Basic answers whatever is asked, in whatever order, and admits defaults plainly. Intermediate ties every answer to a specific requirement or failure mode — and then handles the hardest move: a reviewer connecting two of your own answers that contradict each other.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and a worked example

Basic Version

before the interview:

- list every major piece of your finished capstone (agents, state, RAG, database, API/auth, testing, monitoring, deployment, cost)
- for each piece, be ready to answer "why did you build it that way?"

during the interview:

- answer whatever's asked, in whatever order it comes
- if you don't know something cold, say so plainly instead of guessing out loud as if you were sure

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

build a full-coverage table, one row per subsystem from the build list:

Subsystem	What it is	Requirement it satisfies	Strongest alternative	What the alternative would have cost
-----------	------------	--------------------------	-----------------------	--------------------------------------

do this for ALL of: agents, tools, workflow/LangGraph, state, RAG, database, APIs, login/permissions, testing, monitoring, deployment

during the interview:

- expect the reviewer to jump between rows in any order, including back to a row you already answered, from a different angle
- answer each one the way Doc16 trains: decision, alternative, specific cost, honest trade-off

Worked example row:

Subsystem: Database

What it is: Postgres for structured records, a vector store for RAG chunks

Requirement it satisfies: relational data (users, jobs, billing) needs transactional guarantees a vector store doesn't provide; RAG needs similarity search Postgres alone doesn't do well at this scale

Strongest alternative: one database doing both (e.g. Postgres with a vector extension)

What the alternative would have cost: fewer moving parts to operate, but worse retrieval performance at the stated scale, and coupling two very different access patterns (transactional writes, nearest-neighbor search) into one system's tuning trade-offs

Finding the seams, as a plan:

The same table, with the "genuine decision vs. default" column added, plus a rapid-fire, cross-subsystem exchange to rehearse:

Subsystem	Genuine decision, or default I'd reconsider?
-----------	--

Agents (supervisor pattern)	Genuine -- backward routing on fact-check failure is a real functional requirement a fixed pipeline can't meet
-----------------------------	--

```
Database split (Postgres + vector store) | Genuine -- different access
patterns, named above
Login/permissions (role-based, 3 fixed roles) | DEFAULT -- I built 3
roles because that's what the example in Doc12 used, not because
the requirements specified exactly 3. I'd revisit this against the
actual permission requirements from the live handoff.
Monitoring dashboard layout | DEFAULT -- built what seemed reasonable,
not tied to a specific requirement; see the design_self_review
exercise's exact finding on this
```

Rapid-fire cross-subsystem exchange, to rehearse the "reviewer jumps around and finds a seam" scenario:

```
Reviewer: "You said your database split exists because RAG and
transactional data have different access patterns. But your
deployment design runs both the Postgres instance and the vector
store as the same Docker Compose service, restarted together, with
no independent scaling. Doesn't that undercut the whole reason you
split them?"
```

```
Your move: don't defend the deployment choice as if it were planned
around the database split -- name which one was the real decision
(probably the database split, made for a stated retrieval-latency
requirement) and which one was convenience (single Compose file,
because the stated scale doesn't yet justify separate scaling, and
admit that IS a real limitation if scale grows past what a shared
deployment can handle)
```

Try writing your own full-coverage table plus at least one honest "default, not genuine decision" row, and rehearse a reconciliation for wherever two of your own choices might be in tension, before checking the [Solution](#).

Difference between Basic and Intermediate: Basic plans answers piece by piece. Intermediate plans for cross-examination — the specific requirement behind each piece, the gaps you'll volunteer, and how to concede a real contradiction with a concrete fix.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)